



HEILBRONN UNIVERSITY
OF APPLIED SCIENCES

Universität Heidelberg, Hochschule Heilbronn
MEDIZINISCHE INFORMATIK (MIB)

Bachelor Thesis

Developing a Graphical Interface to Visualize the Results of a K-mer Analysis Based on the System 'GeneStrip'

David Leibensperger

Matr.-Nr.: 208636

July 23, 2024, Heilbronn

Advisor: Prof. Dr.-Ing. Daniel Pfeifer

Second advisor: Prof. Dr. Markus Graf

Statutory Declaration

I herewith declare that I have completed the present thesis independently making use only of the specified literature and aids. Sentences or parts of sentences quoted literally are marked as quotations; identification of other references with regard to the statement and scope of the work is quoted. The thesis in this form or in any other form has not been submitted to an examination body and has not been published.

Heilbronn, July 23, 2024

(Last Name, First Name)

Abstract

In the rapidly evolving field of genomic analysis, this thesis tackles the challenge of making k-mer analysis more accessible to non-expert users. By developing a graphical interface for the GeneStrip system, the project bridges the gap between powerful command-line tools and user-friendly applications. The interface, built with React and Spring Boot, transforms CSV data into interactive tables and graphs, enabling intuitive exploration of k-mer distributions and taxonomic information. This work not only demonstrates the potential for improving bioinformatics tool accessibility but also highlights the importance of user-centered design in scientific software development.

Contents

List of Abbreviations	VI
1 Introduction	1
1.1 Motivation	1
1.2 Problems	1
1.3 Goals	1
2 Background	2
2.1 Bioinformatics	2
2.1.1 K-mers	2
2.1.2 Taxonomy ID	2
2.1.3 FASTA format	3
2.1.4 FASTQ format	3
2.1.5 Genestrip	4
2.2 Frontend Technologies	4
2.2.1 React	4
2.2.2 JSX	6
2.2.3 TypeScript	7
2.2.4 Vite	8
2.2.5 npm	8
2.2.5.1 npm Packages	9
2.3 Backend Technologies	10
2.3.1 Springboot	10
3 System Design	11
3.1 Architecture	11
3.2 Backend	11
3.2.1 Model	12
3.2.1.1 CSV Data	12
3.2.1.2 CSV Service	13
3.2.1.3 File Service	14
3.2.2 Controller	15
3.2.2.1 CSV Controller	15
3.2.2.2 File Controller	16

Contents

3.3	Frontend	17
3.3.1	Components	18
3.3.1.1	Table	18
3.3.1.2	Columns	20
3.3.1.3	Linegraph	20
3.3.2	Utilities	21
3.3.2.1	Api	21
3.3.2.2	TableData	22
3.3.2.3	Utils	22
3.3.3	React App	23
4	Results	24
4.1	Visualization of CSV Data	24
4.2	Visualization of Graphs	27
4.3	User Interface	28
5	Discussion	29
5.1	Analysis	29
5.2	Conclusion	29
5.3	Future Work	29
	Bibliography	30

List of Abbreviations

API Application Programming Interface	9
ASCII American Standard Code for Information Interchange	3
CLI Command-line interface	1
CSV Comma-Separated Values	4
CSS Cascading Style Sheet	22
DOM Document Object Model	4
ES ECMAScript	6
GUI Graphical User Interface	1
HTTP HyperText Transfer Protocol	15
HTML Hypertext Markup Language	6
JSON JavaScript Object Notation	21
JSX JavaScript XML	6
MVC Model-View-Controller	10

List of Abbreviations

NCBI	National Center for Biotechnology Information	2
npm	Node Package Manager	8
REST	Representational State Transfer	10
tax id	Taxonomy ID	2
UI	User Interface	4
URL	Uniform Resource Locator	9

1 Introduction

1.1 Motivation

Ever since the completion of the Human Genome Project in 2003, alongside advancements in fields such as bioinformatics, molecular biology, and computational genomics, gene sequencing technology has seen rapid improvements in terms of cost, performance, and virtually every other aspect. It is widely regarded as one of the most important scientific breakthroughs of the 21st century; consequently, these technological advances have become accessible to the public, with companies now offering analysis of private genomic samples. However, these services do not specifically address the needs related to chronic infections or the analysis of non-human DNA.

1.2 Problems

While tools that utilize k-mer based analysis do exist, they are often clunky and inaccessible to laypeople. Many of these tools are Command-line interface (CLI) based, which can be a hassle for those without a technical background. This complexity can create a barrier for researchers, enthusiasts and anyone who might benefit from k-mer analysis but lack the necessary expertise to navigate these tools.

1.3 Goals

This thesis aims to bridge the gap by developing a Graphical User Interface (GUI) that leverages the command-line based toolkit 'GeneStrip', making advanced genomic analysis more accessible to non-experts. The central goal of this thesis sets is to provide a searchable table as well as graphical visualization of the data provided by 'GeneStrip', so that users can gain deeper insights into their own genomic information.

2 Background

2.1 Bioinformatics

2.1.1 K-mers

A k-mer is a short DNA sequence of length k [1]. For example, if $k=3$, the k-mers of 'CTAG' would be 'CTA' and 'TAG' as seen in Figure 2.1. In metagenomic analysis, k-mers are used to identify and classify organisms by matching these short sequences to known genomes in a database. This allows for rapid and precise analysis of large sets of DNA reads.



Figure 2.1: 3-mers for CTAG

2.1.2 Taxonomy ID

Taxonomy IDs (tax ids) are unique identifiers assigned to each species or taxonomic group in the National Center for Biotechnology Information (NCBI) taxonomy database [2]. For instance, the tax id for humans is 9606 [3]. These tax ids can help in organizing and visualizing evolutionary relationships among different organisms.

2.1.3 FASTA format

FASTA is a text-based format used to encode nucleotide or amino acid sequences as character strings [4]. The first line often contains information like a unique ID and a short description. Although there are no strict formatting rules for the first line, many databases employ standardized naming conventions to enhance data quality and consistency [5].

Listing 2.1: FASTA File Example [6]

```
1 >SSR014849.1 EIXKN4201CFU84 length=93
2 GGGGGGGGGGGGGGGGCTTTTTTGTGTTGGAACCGAAAGG
3 GTTTTGAATTTCAAACCCTTTTCGGTTTCCAACCTTCCAA
4 AGCAATGCCAATA
```

2.1.4 FASTQ format

FASTQ includes precise quality scores for each nucleotide, unlike the sequence-only FASTA format [6]. Instead of FASTA's single-line description, FASTQ files contain four line types.

1. A FASTA-like title line preceeded by an '@' symbol.
2. The sequence data.
3. A termination line beginning with a '+' character, which may be followed by additional data or remain blank.
4. The quality scores for each element in the sequence data are represented by American Standard Code for Information Interchange (ASCII) characters, with values ranging from 0x21 (!) to 0x7e (~).

Listing 2.2: FASTQ File Example [6]

```
1 @SRR014849.1 EIXKN4201CFU84 length=93
2 GGGGGGGGGGGGGGGGCTTTTTTGTGTTGGAACCGAAAGG
3 GTTTTGAATTTCAAACCCTTTTCGGTTTCCAACCTTCCAA
4 AGCAATGCCAATA
5 +SRR014849.1 EIXKN4201CFU84 length=93
6 3+&$#"7F@71, '";C?,B;?6B;:EA1EA
7 1EA5'9B:?:#9EA0D@2EA5':>5?:%A;A8A;?9B;D@
8 /=<?7=9<2A8==
```

2.1.5 Genestrip

Genestrip is a Java-based tool that allows k-mer based metagenomic analysis [7]. Unlike commonly used tools like Kraken, Kraken 2, or KrakenUniq, Genestrip focuses on a smaller memory requirement by utilizing custom databases. To avoid false positives due to shared k-mers among related organisms, Genestrip automatically generates databases from RefSeq and Genbank, enhancing specificity by including species-specific k-mers. Users specify tax ids to tailor databases, facilitating efficient k-mer matching and supporting exact unique k-mer and total k-mer counting akin to KrakenUniq. Genestrip also filters FASTQ files based on database k-mers, significantly reducing file sizes while preserving relevant reads. This approach enables Genestrip to run on less powerful computers, optimizing resource utilization in genomic research. Results are returned in Comma-Separated Values (CSV) format.

2.2 Frontend Technologies

2.2.1 React

React is an open-source JavaScript library developed by Facebook in 2013 [8]. React has established itself as a fundamental tool in modern web development, with over 42% of professional developers having done "extensive development work in over the past year" according to the Stack Overflow Developer Survey 2023 [9]. By using a component-based architecture, React allows developers to break down the User Interface (UI) into reusable, self-contained pieces, making it easier to manage and maintain complex applications. Another key feature of React is its virtual Document Object Model (DOM), as seen in Figure 2.2. Unlike traditional DOM manipulation, the virtual DOM allows React to perform optimized updates. React does this by maintaining a lightweight representation of the actual DOM in memory, the virtual DOM, that allows the DOM to be computed with less overhead.

As seen in Figure 2.2, when the React component tree is first rendered, a virtual DOM is created that represents the actual DOM (Step 0). On subsequent changes, a new virtual DOM is created (Step 1) and is then compared to the original using React's Diffing Algorithm [10]. Provided both virtual DOMs share the same root element, React recursively compares elements until a mismatch is encountered, at which point only that element and its potential children are re-rendered. If the virtual DOMs do not share a root element, the virtual DOMs are deleted, and Step 0 is repeated to create an entirely new virtual DOM. [11]

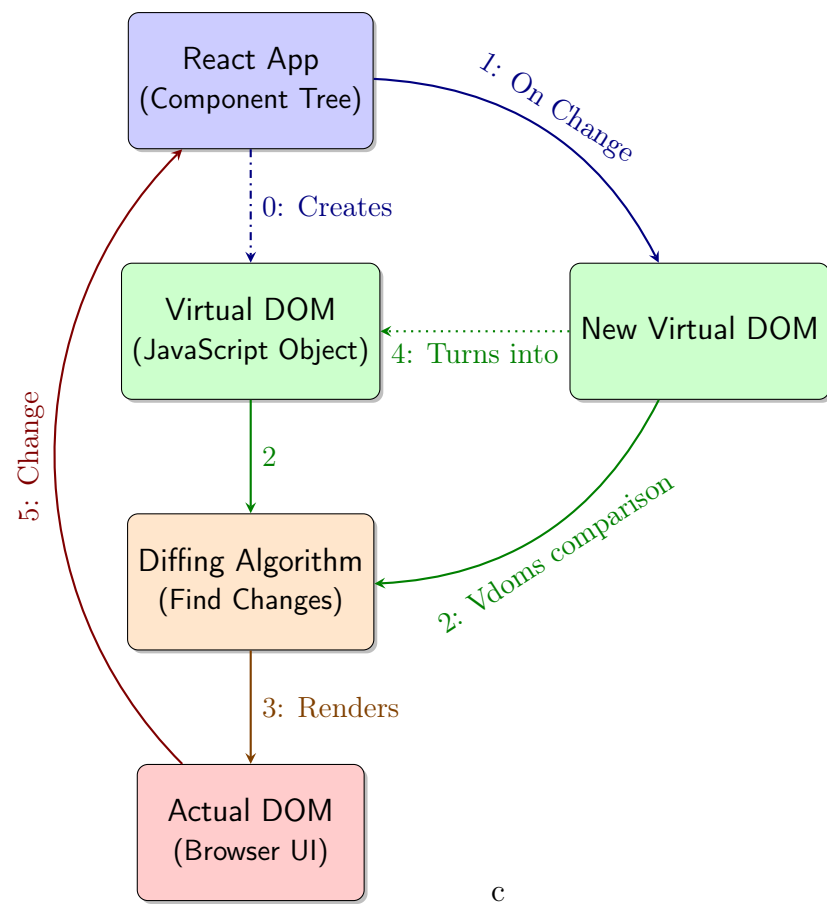


Figure 2.2: React Virtual DOM Workflow

2.2.2 JSX

JavaScript XML (JSX) allows developers to embed Hypertext Markup Language (HTML)-like syntax directly within JavaScript [12].

This integration simplifies the creation of user interfaces. JSX enhances code readability and structure, resembling HTML but ultimately compiling to standard ECMAScript (ES), for browser compatibility. ES is a scripting language specification that JavaScript is based on. Every JSX expression must have a single root element, typically a `div` or a React fragment. This requirement ensures that the compiled JavaScript forms a valid object hierarchy, as JavaScript functions cannot return multiple sibling objects without a parent wrapper.

Listing 2.3: Button with JavaScript

```
1 function createButton(text) {
2   let button = document.createElement('button');
3   button.textContent = text;
4   button.className = 'btn';
5   button.addEventListener('click', () => {
6     console.log('Button_clicked');
7   });
8   return button;
9 }
10 // Example usage
11 let btn = createButton('Click_me');
```

Listing 2.3 is an example for a simple button in JavaScript that executes a function when clicked and can be assigned a name.

Listing 2.4: Button with JSX

```
1 function Button({ text }) {
2   const handleClick = () => {
3     console.log('Button_clicked');
4   };
5   return (
6     <button className="btn" onClick={handleClick}>
7       {text}
8     </button>
9   );
10 }
11 // Example usage
12 <Button text="Click_me" />
```

The same functionality implemented with JSX and Typescript in Listing 2.4 shows a reusable component with improved maintainability as well as readability can be created.

2.2.3 TypeScript

TypeScript is a statically typed superset of JavaScript [13]. It enables developers to define types for variables, function parameters and return values. This helps catch errors early during development and enhances code quality and maintainability. TypeScript seamlessly integrates with modern Web frameworks and libraries, like React, Angular, or Vue.js, allowing developers to leverage its features without forcing them to use a specific framework.

Listing 2.5: JavaScript Code

```
1 // JavaScript
2 function calculateArea(width, height) {
3     return width * height;
4 }
5
6 // Example usage
7 let area = calculateArea(5, '10'); // '10' is a string, not a number
```

Listing 2.5 shows valid JavaScript code that will execute without any errors during compilation. However, if a string is passed instead of a number for either width or height, the function will still execute, but it will produce unexpected results. In this case, a string concatenation ('510') instead of multiplication (50).

Listing 2.6: TypeScript Code

```
1 // TypeScript
2 function calculateArea(width: number, height: number): number {
3     return width * height;
4 }
5
6 // Example usage
7 let area: number = calculateArea(5, '10');
8 // Error: Argument of type 'string' is not assignable
9 // to parameter of type 'number'
```

If TypeScript is used like in Listing 2.6, to solve the same problem instead of JavaScript, the potential error is already caught at compile time.

2.2.4 Vite

Vite is a build tool that focuses on fast development and optimized builds for frontend web projects [14]. It leverages native ES modules to achieve faster bundling and serving of files, resulting in significantly improved performance compared to traditional bundlers. Vite's architecture is optimized for rapid development cycles, offering near-instantaneous server startup and hot module replacement that updates changes in real-time without requiring a full page refresh. One of Vite's key features is its out-of-the-box support for TypeScript and JSX, among other contemporary JavaScript features. Vite focuses on compilation rather than type checking for TypeScript files. This approach prioritizes speed during development, leaving type checking to be handled separately by the IDE or build process. Vite's efficiency extends beyond the development phase. It employs rollup.js for production builds, resulting in highly optimized and lean output [15]. Using ES modules for development and a more traditional bundling strategy for production allows Vite to offer a lightning-fast development experience as well as optimized production performance.

2.2.5 npm

Node Package Manager (npm) is a critical component of the JavaScript ecosystem, hosting millions of packages, ranging from utility functions to comprehensive frameworks like React and Angular [16]. As a package manager, npm facilitates the installation, versioning, and updating of packages, simplifying the process of incorporating third-party libraries into projects. When *npm install <package-name>* is run, it fetches and installs the package from the npm registry. The *package.json* file lists your project's metadata and dependencies. Dependencies are categorized into 'dependencies' for runtime and 'devDependencies' for development. The *package-lock.json* file locks specific versions of installed packages, ensuring consistency across different environments. This file is auto-generated during installation. npm manages dependencies using semantic versioning (semver), handling version ranges specified in *package.json* [17]. Running *npm update* updates packages to the latest versions within specified ranges. *npm outdated* checks for newer versions of installed packages. This system ensures reliable and reproducible dependency management.

2.2.5.1 npm Packages

Below are the most important npm packages used in this project.

React Router

React Router is a library that allows for routing in React, enabling developers to build single-page applications with dynamic navigation [18]. It simplifies the management of browser history, Uniform Resource Locator (URL) parameters, and dynamic routing with a declarative Application Programming Interface (API) that fits well within the React ecosystem.

Recharts

Recharts provides a comprehensive suite of components for integrating interactive and visually appealing charts within web applications [19]. These components cover a wide range of chart types, including line charts, bar charts, pie charts, area charts, and radar charts. Built with React and D3.js, Recharts is also highly responsive and performant. The library's robust API supports extensive customization, adding features such as animations, tooltips, and legends. With thorough and simple documentation Recharts, was chosen for this project over a more complex framework like visx [20].

TanStack Table

TanStack Table is a lightweight table framework with a headless approach to design, allowing for full customizability [21]. The built in functionalities for sorting, grouping, pagination allow for quick and effective table implementations, that can be as simple or complex as the project requires.

2.3 Backend Technologies

2.3.1 Springboot

Spring Boot is a Java-based open-source framework built on top of Spring [22]. It excels in creating Representational State Transfer (REST)ful APIs and backend services by providing out-of-the-box features such as embedded servers, auto-configuration, and production-ready tools. Spring Boot promotes convention over configuration, reducing boilerplate code and allowing developers to focus more on business logic. Spring Boot applications often adhere to the Model-View-Controller (MVC) design pattern [23].

The MVC is a software architectural pattern that separates an application into three main logical components as seen in Figure 2.3.

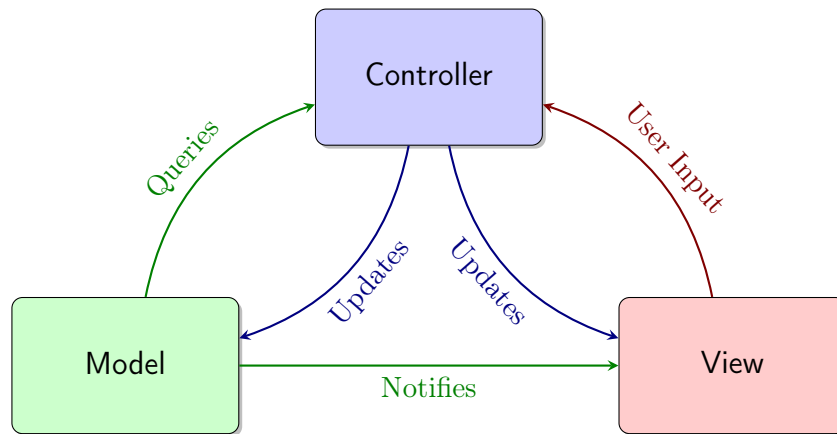


Figure 2.3: Model-View-Controller pattern

1. Model
 - The Model deals with the application logic and data abstraction.
2. View
 - The View renders the data from Model in the Browser as HTML and displays it in the Browser.
3. Controller
 - The Controller updates the Model according to changes in the View.

3 System Design

3.1 Architecture

The system loosely adheres to a MVC design approach. The model and controller are implemented in a Springboot backend while the view component is handled by a React frontend.

3.2 Backend

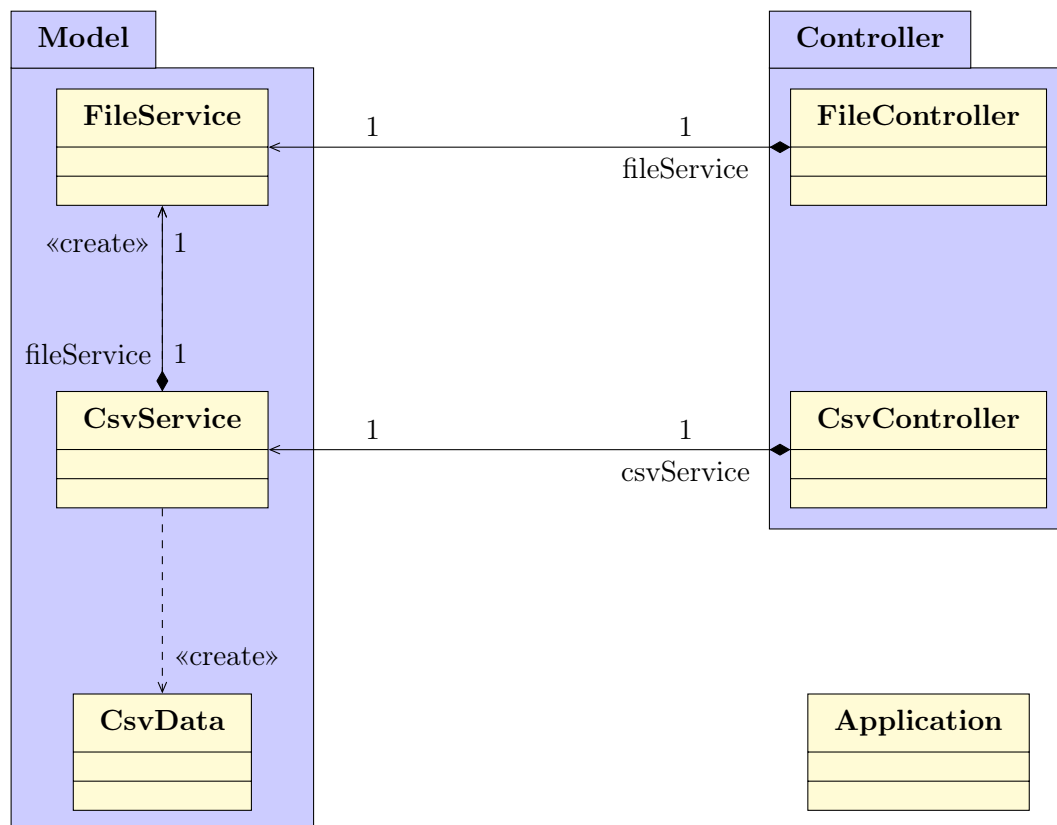


Figure 3.1: Backend Uml

3.2.1 Model

3.2.1.1 CSV Data

CsvData is designed to encapsulate the data that is parsed from the CSV files provided by Genestrip. Each CsvData object represents a single line of the original CSV file. Each value is stored in its own private data field, which can be interacted with through conventional getter and setter methods which have been omitted in Figure 3.2 to avoid repetition.

Below is an overview of the most relevant fields. For a more detailed explanation, please refer to the GeneStrip Documentation [7].

CsvData.java	
- name : String - rank : String - taxId : int - reads : int - kmersFromReads : int - kmers : int - uniqueKmers : int - contigs : int - averageContigLength : double - maxContigLength : int - maxContigDesc : String - dbCoverage : double - normalizedKmers : double - expUniqueKmers : double - uniqueKmersOverExp : double - qualityPrediction : double - maxKmerCounts : int[]	<i>name</i>: The tax id name. <i>rank</i>: The tax id rank. <i>taxid</i>: The tax id. <i>reads</i>: The number of reads for the tax id. <i>kmers from reads</i>: The number of k-mers from classified reads which are consistent with the read's tax id. <i>kmers</i>: All matched k-mers specific to the tax id's genome according to the database. <i>unique kmers</i>: All unique k-mers specific to the tax id's genome, counted once. <i>quality prediction</i>: Computed as $normalizedkmers * uniquekmers/exp..$ <i>max kmer counts</i>: Frequencies of the most frequent unique k-mers specific to the tax id's genome in descending order.
+ CsvData() + getterMethods() + setterMethods()	

Figure 3.2: CsvData

3.2.1.2 CSV Service

CsvService is designed to read and parse multiple CSV files from a set directory, store the data in a nested list structure, and provide methods to access the parsed data.

CsvService.java
- fileService : FileService - csvDataList : List<List<CsvData>>
+ CsvService() + readCsvDirectory() : void + readCsvFile(File) : List<CsvData> + getCsvDataByFilepath(String) : List<CsvData> + getName(int, int) : String + get... + getMaxKmerCounts(int, int) : int[] + getCsvDataList() : List<List<CsvData>> + getSize() : int + getNestedSize(int) : int

Figure 3.3: CsvService

csvDataList

This nested list stores the CSV files. The inner list represents the line number of a given entry within its CSV file, while the outer list represents the index for the CSV file itself.

readCsvFile(File)

This method parses each CSV file into a *List<CsvData>* object using Java's Scanner class, processing each file line by line. It ensures that the correct data types are used and handles the optional *maxKmerCounts* property for each line.

getCsvDataByFilepath(String) This method calls *readCsvFile* with the given filepath as input.

getName(int, int) This method retrieves data from *csvDataList*. For example, to get the name associated with the seventh tax id in the third file, you would call *getName(3, 7)*. Similar methods are implemented for all fields of *CsvData* but are omitted in Figure 3.3 for brevity.

getCsvDataList() This method returns *csvDataList*.

getSize & *getNestedSize* These methods return the size of *csvDataList* and the size of a specific CSV file, respectively.

3.2.1.3 File Service

FileService reads the CSV files from a set directory.

FileService.java
- files : List<File>
+ FileService() + getFiles() : List<File>

files: A list with the filepaths of the CSV files within the set directory.

getFiles(): This method returns a list containing all CSV files within the set directory.

Figure 3.4: FileService

3.2.2 Controller

3.2.2.1 CSV Controller

CsvController is a Spring Boot REST controller that handles the HyperText Transfer Protocol (HTTP) requests to interact with CSV files through a service layer. Each CsvData field is individually accessible through the API with custom URL parameters for the specific CSV file (index), and line (index 2) the field data was parsed from.

CsvController.java
- csvService : CsvService
+ CsvController(CsvService) + @GetMapping("/name/index/index2") : ResponseEntity<String> + @Get... + @GetMapping("/maxKmerCounts/index/index2") : ResponseEntity<int[]> + @GetMapping("/table") : ResponseEntity<List<CsvData> > + @GetMapping("/tables") : ResponseEntity<List<List<CsvData> > > + @GetMapping("/size") : int

Figure 3.5: CsvController

<i>@GetMapping("/name/index/index2")</i>	This method handles GET requests to "/name/index/index2", checks if the indices are valid in csvService, and returns the corresponding name if they are. If the indices are out of bounds, a 404 error is returned.
<i>@GetMapping("/table")</i>	This method returns the CSV data for a given filepath.
<i>@GetMapping("/tables")</i>	This method returns the CSV data for all files.
<i>@GetMapping("/size")</i>	This method returns the amount of CSV files.

3.2.2.2 File Controller

FileController.java
- fileService : FileService
+ FileController(FileService) + @GetMapping("/all")

FileController returns a list of all currently present CSV files.

Figure 3.6: FileController

3.3 Frontend

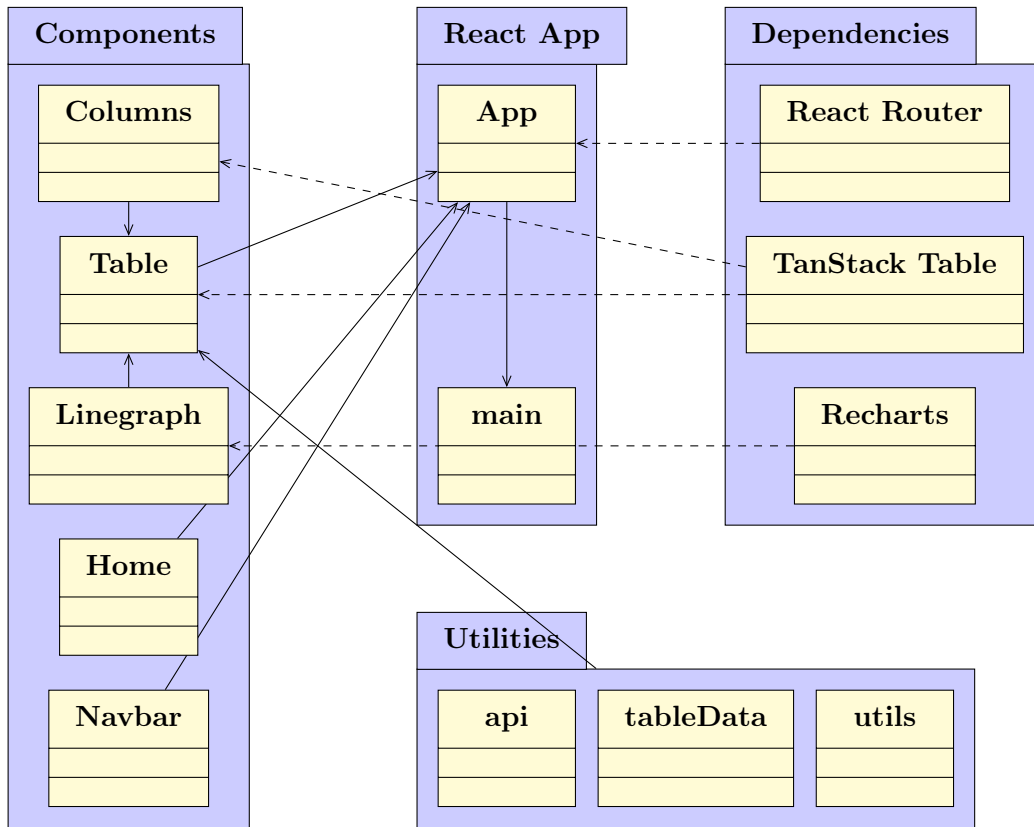


Figure 3.7: Frontend uml

3.3.1 Components

3.3.1.1 Table

Table is a custom React component that implements the graphical representation of CSV files in the Browser.

Table.tsx
- files: string[] - isGraphVisible: boolean - selectedFile: string - data: TableData[] - globalFilter: string - coloringMethod: 'none' 'median' 'mean' - selectedRow: TableData null
+ toggleGraph(row: TableData): void + handleClear(): void + getCellColor(value: number null undefined): string + useEffect() + useMemo()

Figure 3.8: Table.tsx Component

<i>files</i>	This array contains the CSV filepaths as strings.
<i>isGraphVisible</i>	This boolean is used to conditionally render the Linegraph component for a given row.
<i>data</i>	This array contains the currently displayed CSV file data.
<i>selectedFile</i>	This string represents the filepath of the currently selected CSV file.
<i>globalFilter</i>	This string contains the current table search term, if present.

<i>coloringMethod</i>	This contains the currently selected coloring method.
<i>selectedRow</i>	This contains the data of the currently selected row.
<i>toggleGraph(row: TableData)</i>	This function toggles <i>isGraphVisible</i> depending on if the selected row contains the <i>maxKmerCounts</i> property.
<i>handleClear()</i>	This function resets the search field and <i>globalFilter</i> .
<i>getCellColor(value: number / null / undefined)</i>	This function sets the row color based on the <i>quality prediction</i> value for that row.
<i>useEffect()</i>	A <i>useEffect</i> hook is used to fetch the first file from the FileService Section 3.2.1.3 method <i>getFiles</i> as the initial CSV to be displayed. A second hook fetches the appropriate CSV file when it changes.
<i>useMemo()</i>	A <i>useMemo</i> hook sets the <i>coloringMethod</i> property according to the mean or median of the <i>qualityPrediction</i> value of each row.

3.3.1.2 Columns

Columns.tsx defines table columns using React Table. The *getColumns()* function, using *getCellColor* and *toggleGraph*, links columns to *TableData* properties, displaying values, "N/A", or a button for *toggleGraph*. The *qualityPrediction* column applies a CSS class based on its value.

Columns.tsx
+ getColumns(getCellColor: Function, toggleGraph: Function): Column[]

Figure 3.9: Columns.tsx Component

3.3.1.3 Linegraph

Linegraph.tsx implements the graphical representation of the maxKmerCounts property for a given CSV entry. When implemented as a JSX component, it takes *maxKmerCounts* and *name* as props to render a graph.

Linegraph.tsx
- maxKmerCounts: number[] - name: string
+ Linegraph(maxKmerCounts: number[], name: string)

Figure 3.10: Linegraph.tsx Component

3.3.2 Utilities

3.3.2.1 Api

The `api.ts` file contains two asynchronous functions to fetch data from a server. The *fetchFiles* function retrieves the list of CSV file names by making a GET request to "http://localhost:8080/api/file/all" and returns the response as a JavaScript Object Notation (JSON) array of strings. The *fetchTableData* function takes a file path as an argument, makes a GET request to "http://localhost:8080/api/csv/table" with the file path as a query parameter, and returns the response as a JSON array of `TableData` objects.

api.ts
+ fetchFiles(): Promise<string[]> + fetchTableData(filePath: string): Promise<TableData[]>

Figure 3.11: `api.ts`

3.3.2.2 TableData

tableData.ts
- name: string - rank: string - taxId: string - reads: number - kmersFromReads: number - kmers: number - uniqueKmers: number - contigs: number - averageContigLength: number - maxContigLength: number - maxContigDesc: string - dbCoverage: number - normalizedKmers: number - expUniqueKmers: number - uniqueKmersOverExp: number - qualityPrediction: number - maxKmerCounts: number[]

TableData provides an interface, mirroring the fields from CsvData 3.2.1.1, to ensure proper type safety.

Figure 3.12: tableData.js

3.3.2.3 Utils

The utils.ts file includes two functions: *calculateMedianAndMean*, which calculates the median and mean of *qualityPrediction* values in a *TableData* array, and *getCellColor*, which assigns a Cascading Style Sheet (CSS) class based on a cell's value and a specified *coloringMethod*.

utils.ts
+ calculateMedianAndMean(data: TableData[]): median: number, mean: number + getCellColor(value: number null undefined, coloringMethod: "none" "median" "mean", compareValue: number): string

Figure 3.13: utils.js Class

3.3.3 React App

App.tsx
+ App(): JSX.Element

Figure 3.14: App Component

main.tsx
+ createRoot(): ReactDOM.root

Figure 3.15: main Component

The *createRoot* function is used to initialize and manage the rendering of a React application [24]. In the *main.tsx* file, *createRoot* targets an element with the id *root* in the *index.html* file, where the application will be mounted. The App Component contains the browsing logic that returns a given Component, like *Table.tsx*, alongside a basic navigation bar *Navbar.tsx* wrapped in a JSX fragment.

4 Results

4.1 Visualization of CSV Data

The CSV data generated by Genestrip is visualized in the Table Component. As illustrated in Figure 4.1, the search bar enables users to search through the table using all attributes. The search functionality is persistent across different CSV files, allowing for quicker searches for a specific virus across multiple files.

Name ↓↑	Rank ↓↑	Tax ID ↓↑	Reads ↓↑	Kmers From Reads ↓↑	Kmers ↓↑
Borrelia coriaceae	species	144	0	0	1
Borrelia coriaceae ATCC 43381	strain	1408429	0	0	5

Figure 4.1: Table searchbar

4 Results

The coloring option, shown in Figure 4.2, visually highlights rows based on their *qualityPrediction* value. This feature allows users to quickly distinguish desirable rows with better matches from those with lower quality predictions.

Color by median ▾

Average Contig Length	Max Contig Length	Max Contig Description	dbCoverage	Normalized Kmers
0	0	0	0	0
37.71549987792969	305	@SRR17281117.346082.1 346082 length=446	0.2232999950647354	994015232
38.172401428222656	61	@SRR17281117.1076143.1 1076143 length=982	0.005799999926239252	255051.84375
31.5	32	@SRR17281117.720340.1 720340 length=452	0	433.7008056640625

Figure 4.2: Table coloring

The various CSV files can be selected as demonstrated in Figure 4.3. Upon initially loading the page, the first CSV file from the specified directory is displayed, ensuring that users do not encounter an empty page. Additionally, the *Toggle Graph* button is available, which enables the display of a frequency graph for the k-mers of a given row as shown in Figure 4.4.

/home/beluzzi/Code/geneback/csv/borrelia_plasmid_matchlr_tick6.csv
▼

Normalized Kmers ↓↑	Expected Unique Kmers ↓↑	Unique Kmers Over Expected ↓↑	Quality Prediction ↓↑	Actions ↓↑
3769564.5	139514	0.863099992275238	3253631.7500	Toggle Graph

Figure 4.3: Table file selection

4.2 Visualization of Graphs

The *maxKmerCounts* property is rendered as a frequency graph, with the graph labeled using the *name* property from the corresponding row, as depicted in Figure 4.4.

Most frequent k-mers for Human gammaherpesvirus 4

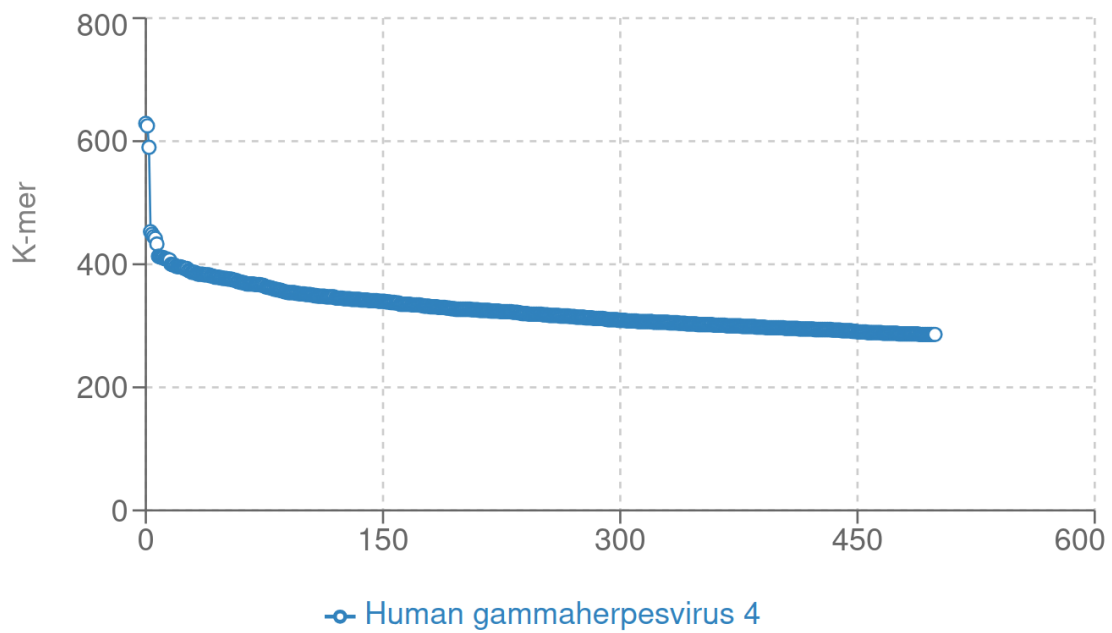


Figure 4.4: Linegraph

4.3 User Interface

A simple home page was included to test routing functionality and provide an entry point for future updates to the UI. These updates could include documentation for Genestrip, a user management system, or other new features.

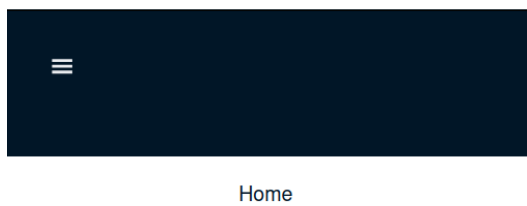


Figure 4.5: Home page&Navbar closed

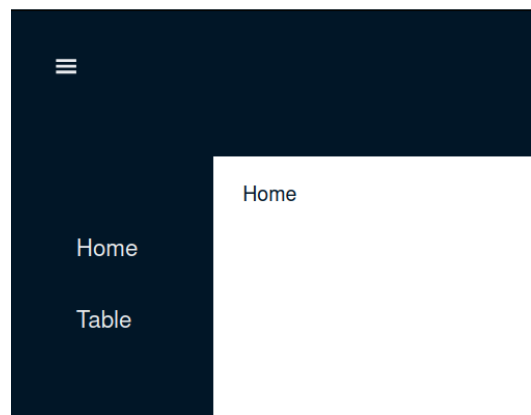


Figure 4.6: Home page&Navbar open

5 Discussion

5.1 Analysis

The primary goal of this thesis was to provide easier access for non-technical users to CSV data from CLI-only tools. While this goal was broadly achieved, the process revealed the importance of addressing more fundamental aspects such as the API design and overall structure. Consequently, much work remains to be done to implement desirable features in the frontend. Achieving this is particularly challenging without feedback or a clear vision from end users.

5.2 Conclusion

This project represents a step towards enabling laypeople to easily interface with medical data that holds personal significance. While some progress has been made, it is clear that actual user reviews and feedback are crucial to enhancing the overall utility of this tool. User input would be invaluable in refining and improving its functionality and user experience.

5.3 Future Work

Significant security enhancements are necessary to make this tool suitable for hosting. The next logical step involves proper integration with GeneStrip, followed by UI polishing and expanding menu functionality. Additionally, it is essential to address and resolve existing issues in the UI. Potentially migrating more components to frameworks such as TanStack or Next.js could also be beneficial. Overall, continued development and user feedback will be key to the evolution and success of this project.

Bibliography

- [1] *K-Mer Counting, Part I: Introduction* | BioInfoLogics. URL: <https://bioinfologics.github.io/post/2018/09/17/k-mer-counting-part-i-introduction/> (visited on 07/16/2024).
- [2] *Home - Taxonomy - NCBI*. URL: <https://www.ncbi.nlm.nih.gov/taxonomy> (visited on 07/20/2024).
- [3] *Taxonomy Browser (Homo Sapiens)*. URL: <https://www.ncbi.nlm.nih.gov/Taxonomy/Browser/wwwtax.cgi?mode=Info&id=9606&lvl=3&lin=f&keep=1&srchmode=1&unlock> (visited on 07/21/2024).
- [4] W R Pearson and D J Lipman. “Improved Tools for Biological Sequence Comparison.” In: *Proceedings of the National Academy of Sciences of the United States of America* 85.8 (Apr. 1988), pp. 2444–2448. ISSN: 0027-8424. pmid: 3162770. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC280013/> (visited on 07/20/2024).
- [5] *FASTA Format for Nucleotide Sequences*. URL: <https://www.ncbi.nlm.nih.gov/genbank/fastaformat/> (visited on 07/20/2024).
- [6] Peter J. A. Cock et al. “The Sanger FASTQ File Format for Sequences with Quality Scores, and the Solexa/Illumina FASTQ Variants”. In: *Nucleic Acids Research* 38.6 (Apr. 2010), pp. 1767–1771. ISSN: 0305-1048. DOI: 10.1093/nar/gkp1137. pmid: 20015970. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC2847217/> (visited on 07/20/2024).
- [7] pfeiferd. *Pfeiferd/Genestrip*. July 10, 2024. URL: <https://github.com/pfeiferd/genestrip> (visited on 07/14/2024).
- [8] *Quick Start – React*. URL: <https://react.dev/learn> (visited on 07/16/2024).
- [9] *Stack Overflow Developer Survey 2023*. Stack Overflow. URL: https://survey.stackoverflow.co/2023/?utm_source=social-share&utm_medium=social&utm_campaign=dev-survey-2023 (visited on 07/17/2024).
- [10] *Reconciliation – React*. URL: <https://legacy.reactjs.org/docs/reconciliation.html> (visited on 07/20/2024).
- [11] Ibadehin Mojeed. *What Is the Virtual DOM in React?* LogRocket Blog. Aug. 16, 2022. URL: <https://blog.logrocket.com/virtual-dom-react/> (visited on 07/17/2024).

- [12] *JSX*. URL: <https://facebook.github.io/jsx/> (visited on 07/17/2024).
- [13] *JavaScript With Syntax For Types*. URL: <https://www.typescriptlang.org/> (visited on 07/16/2024).
- [14] *Vite*. vitejs. URL: <https://vitejs.dev> (visited on 07/16/2024).
- [15] *Rollup*. URL: <https://rollupjs.org/> (visited on 07/23/2024).
- [16] *About Npm / Npm Docs*. URL: <https://docs.npmjs.com/about-npm> (visited on 07/16/2024).
- [17] *About Semantic Versioning / Npm Docs*. URL: <https://docs.npmjs.com/about-semantic-versioning> (visited on 07/23/2024).
- [18] *React-Router*. npm. July 16, 2024. URL: <https://www.npmjs.com/package/react-router> (visited on 07/16/2024).
- [19] *Recharts*. npm. May 8, 2024. URL: <https://www.npmjs.com/package/recharts> (visited on 07/16/2024).
- [20] *Visx / Visualization Components*. URL: <https://airbnb.io/visx/> (visited on 07/16/2024).
- [21] *@tanstack/React-Table*. npm. July 14, 2024. URL: <https://www.npmjs.com/package/@tanstack/react-table> (visited on 07/16/2024).
- [22] *Spring Boot*. Spring Boot. URL: <https://spring.io/projects/spring-boot> (visited on 07/16/2024).
- [23] *Spring MVC :: Spring Boot*. URL: <https://docs.spring.io/spring-boot/how-to/spring-mvc.html> (visited on 07/23/2024).
- [24] *createRoot – React*. URL: <https://react.dev/reference/react-dom/client/createRoot> (visited on 07/23/2024).
- [25] Conrad L Schoch et al. “NCBI Taxonomy: A Comprehensive Update on Curation, Resources and Tools”. In: *Database* 2020 (Jan. 1, 2020), baaa062. ISSN: 1758-0463. DOI: 10.1093/database/baaa062. URL: <https://academic.oup.com/database/article/doi/10.1093/database/baaa062/5881509> (visited on 07/21/2024).
- [26] *Writing Markup with JSX – React*. URL: <https://react.dev/learn/writing-markup-with-jsx> (visited on 07/16/2024).